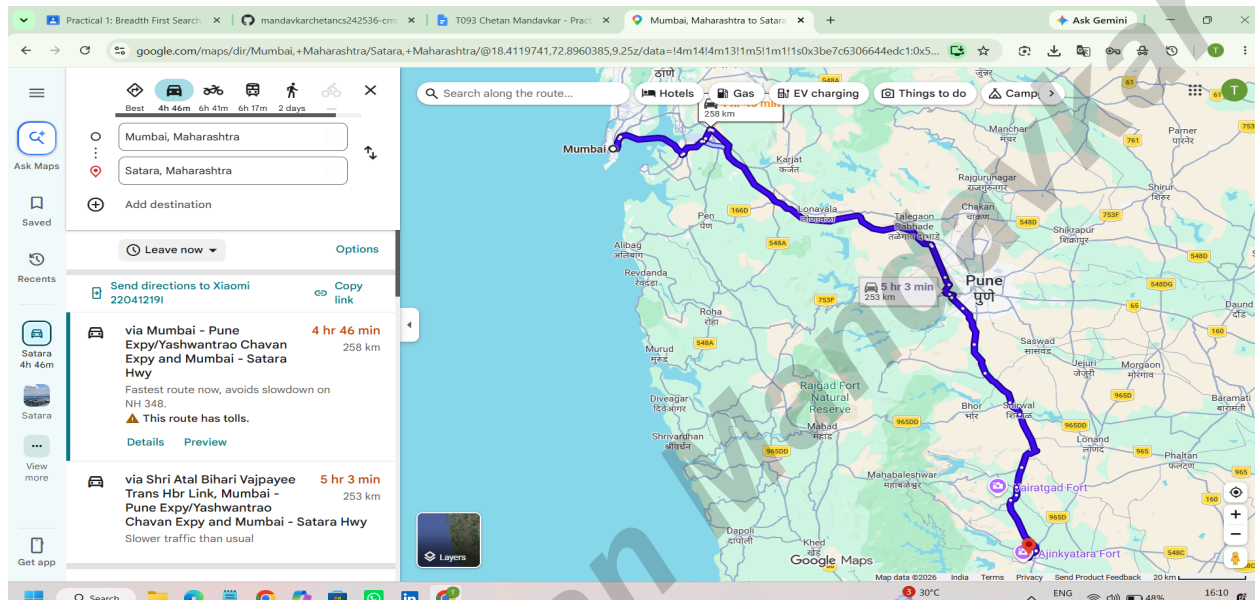


Sheth L.U.J. And Sir M.V. College

Aim:- A* Search and Recursive Best-First Search

A] Implement the A* Search algorithm for solving a pathfinding problem.

INPUT:-



```
AI Practical No. 2.py - C:\Users\UT-46\OneDrive\Documents\AI Practical No. 2.py (3,14,6)
File Edit Format Run Options Window Help
import heapq

graph = {
    "Mumbai": {
        "Lonavala": 85,
        "Nashik": 165,
        "Alibag": 154
    },
    "Lonavala": {
        "Mumbai": 85,
        "Pune": 65
    },
    "Nashik": {
        "Mumbai": 165,
        "Satara": 210
    },
    "Alibag": {
        "Mumbai": 154
    },
    "Pune": {

```

Name:- Chetan Mandavkar

Roll No. T093

Subject:- Artificial Intelligence

Sheth L.U.J. And Sir M.V. College

```
AI Practical No. 2.py - C:\Users\IT-46\OneDrive\Documents\AI Practical No. 2.py (3.14.6)
File Edit Format Run Options Window Help

"Pune": {
    "Lonavala": 65,
    "Satara": 110
},

"Satara": {}

}

heuristic = {
    "Mumbai": 260,
    "Lonavala": 175,
    "Nashik": 210,
    "Alibag": 300,
    "Pune": 110,
    "Satara": 0
}

def a_star(start, goal):
    priority_queue = []
    heapq.heappush(priority_queue, (heuristic[start], 0, start, [start]))

    visited = set()
```

```
AI Practical No. 2.py - C:\Users\IT-46\OneDrive\Documents\AI Practical No. 2.py (3.14.6)
File Edit Format Run Options Window Help

while priority_queue:
    f, g, current, path = heapq.heappop(priority_queue)

    if current == goal:
        return path, g

    if current in visited:
        continue

    visited.add(current)

    for neighbor, distance in graph[current].items():
        if neighbor not in visited:
            new_g = g + distance
            new_f = new_g + heuristic[neighbor]

            heapq.heappush(
                priority_queue,
                (new_f, new_g, neighbor, path + [neighbor])
            )

    return None, float("inf")
```

Name:- Chetan Mandavkar

Roll No. T093

Subject:- Artificial Intelligence

Sheth L.U.J. And Sir M.V. College

```
AI Practical No. 2.py - C:\Users\IT-46\OneDrive\Documents\AI Practical No. 2.py (3,14,6)
File Edit Format Run Options Window Help

for neighbor, distance in graph[current].items():

    if neighbor not in visited:

        new_g = g + distance
        new_f = new_g + heuristic[neighbor]

        heapq.heappush(
            priority_queue,
            (new_f, new_g, neighbor, path + [neighbor])
        )

    return None, float("inf")

start_city = "Mumbai"
goal_city = "Satara"

path, cost = a_star(start_city, goal_city)

print("==== A* Search ====")
print("Route Found :", " -> ".join(path))
print("Total Distance :", cost, "km")

Ln: 85 Col: 0
30°C Mostly cloudy 11:02 17-07-2026
```

OUTPUT:-

```
>>>
===== RESTART: C:\Users\IT-46\OneDrive\Documents\AI Practical No. 2.py =====
==== A* Search ====
Route Found : Mumbai -> Lonavala -> Pune -> Satara
Total Distance : 260 km

Type here to search 30°C Mostly cloudy 11:03 17-07-2026
```

Name:- Chetan Mandavkar

Roll No. T093

Subject:- Artificial Intelligence

Sheth L.U.J. And Sir M.V. College

Graph Plotting:-

```
AI Practical No. 2.py - C:\Users\IT-46\OneDrive\Documents\AI Practical No. 2.py (3.14.6)
File Edit Format Run Options Window Help
# -----
# GRAPH PLOTTING
# -----

import heapq
import matplotlib.pyplot as plt
import networkx as nx

node_positions = {
    "Mumbai": (2, 8),
    "Lonavala": (1, 6),
    "Nashik": (3, 6),
    "Alibag": (1, 4),
    "Pune": (3, 4),
    "Satara": (2, 2)
}

G = nx.DiGraph()

for city, neighbors in graph.items():
    for neighbor, distance in neighbors.items():
        G.add_edge(city, neighbor, weight=distance)
```

```
AI Practical No. 2.py - C:\Users\IT-46\OneDrive\Documents\AI Practical No. 2.py (3.14.6)
File Edit Format Run Options Window Help
plt.figure(figsize=(10,8))

path_edges = list(zip(path, path[1:]))

normal_edges = [edge for edge in G.edges() if edge not in path_edges]

nx.draw_networkx_nodes(
    G,
    node_positions,
    node_size=2600,
    node_color="lightblue"
)

nx.draw_networkx_edges(
    G,
    node_positions,
    edgelist=normal_edges,
    width=1.5,
    edge_color="gray",
    arrows=True,
    arrowsize=18
```

Name:- Chetan Mandavkar

Roll No. T093

Subject:- Artificial Intelligence

Sheth L.U.J. And Sir M.V. College

```
AI Practical No. 2.py - C:\Users\IT-46\OneDrive\Documents\AI Practical No. 2.py (3.14.6)
File Edit Format Run Options Window Help
    arrowsize=18
)

nx.draw_networkx_edges(
    G,
    node_positions,
    edgelist=path_edges,
    width=3.5,
    edge_color="darkorange",
    arrows=True,
    arrowsize=18
)

labels = {
    city: f"{city}\nh(n)={heuristic[city]}"
    for city in G.nodes()
}

nx.draw_networkx_labels(
    G,
    node_positions,
    labels,
    font_size=9
)
Ln: 85 Col: 0
```

```
AI Practical No. 2.py - C:\Users\IT-46\OneDrive\Documents\AI Practical No. 2.py (3.14.6)
File Edit Format Run Options Window Help
nx.draw_networkx_labels(
    G,
    node_positions,
    labels,
    font_size=9,
    font_weight="bold"
)

edge_labels = nx.get_edge_attributes(G, "weight")
edge_labels = {
    edge: f"{distance} km"
    for edge, distance in edge_labels.items()
}

nx.draw_networkx_edge_labels(
    G,
    node_positions,
    edge_labels=edge_labels,
    font_color="red",
    font_size=9
)

plt.title(
    "A* Routing Map: Mumbai to Satara\n"
)
Ln: 85 Col: 0
```

Name:- Chetan Mandavkar

Roll No. T093

Subject:- Artificial Intelligence

Sheth L.U.J. And Sir M.V. College

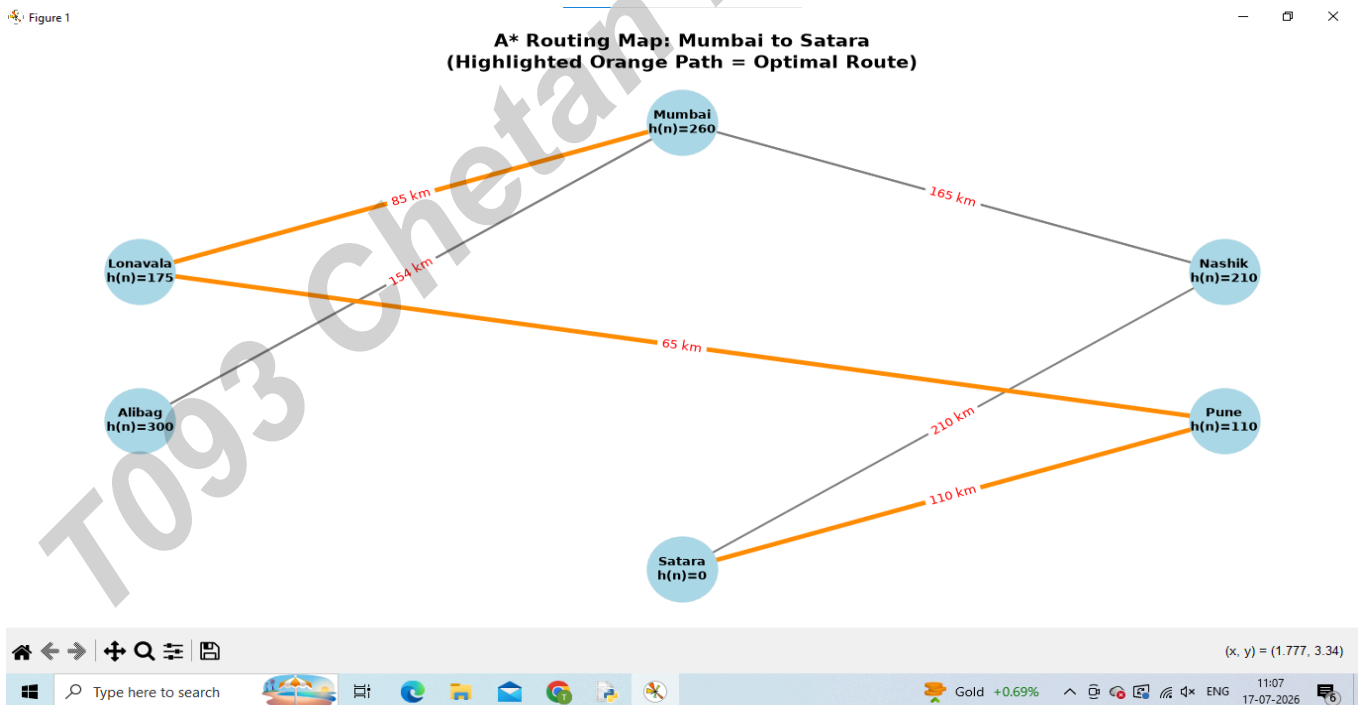
```
AI Practical No. 2.py - C:\Users\UT-46\OneDrive\Documents\AI Practical No. 2.py (3.14.6)
File Edit Format Run Options Window Help
edge_labels = nx.get_edge_attributes(G, 'weight')
edge_labels = {
    edge: f"{distance} km"
    for edge, distance in edge_labels.items()
}

nx.draw_networkx_edge_labels(
    G,
    node_positions,
    edge_labels=edge_labels,
    font_color="red",
    font_size=9
)

plt.title(
    "A* Routing Map: Mumbai to Satara\n"
    "(Highlighted Orange Path = Optimal Route)",
    fontsize=14,
    fontweight="bold"
)

plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT:-



Name:- Chetan Mandavkar

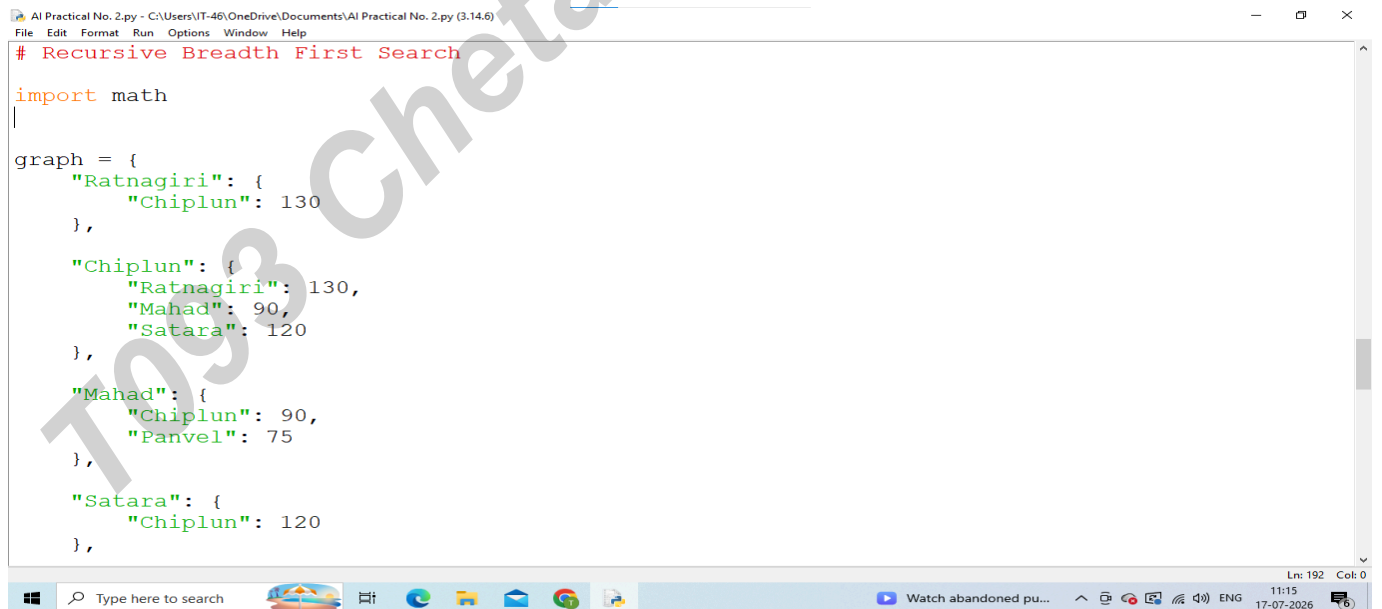
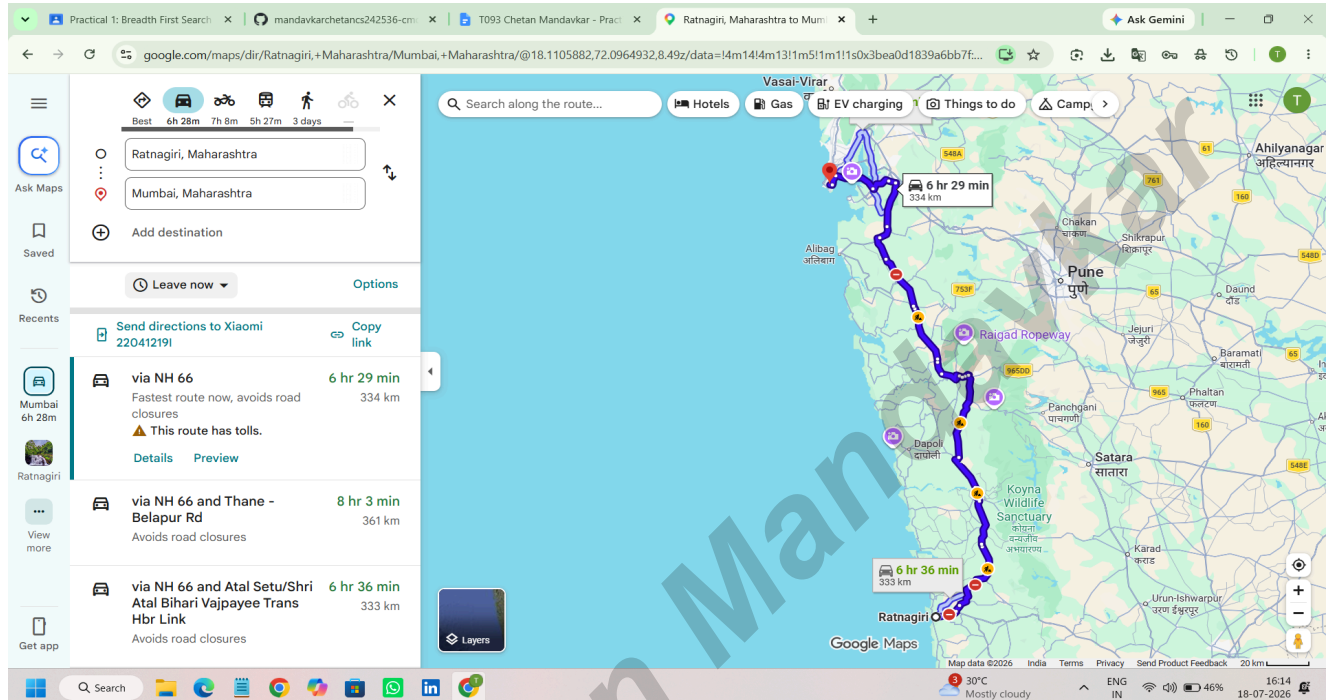
Roll No. T093

Subject:- Artificial Intelligence

Sheth L.U.J. And Sir M.V. College

B] Implement the Recursive Best-First Search algorithm for the same problem.

INPUT:-



Name:- Chetan Mandavkar

Roll No. T093

Subject:- Artificial Intelligence

Sheth L.U.J. And Sir M.V. College

```
AI Practical No. 2.py - C:\Users\IT-46\OneDrive\Documents\AI Practical No. 2.py (3.14.6)
File Edit Format Run Options Window Help

"Panvel": {
    "Mahad": 75,
    "Mumbai": 40
},
"Mumbai": {}
}

heuristic = {
    "Ratnagiri": 300,
    "Chiplun": 220,
    "Mahad": 110,
    "Satara": 250,
    "Panvel": 40,
    "Mumbai": 0
}

def rbfs(node, goal, g_cost, f_limit, path):
    if node == goal:
        return path, g_cost
```

Ln: 192 Col: 0
11:16
17-07-2026

```
AI Practical No. 2.py - C:\Users\IT-46\OneDrive\Documents\AI Practical No. 2.py (3.14.6)
File Edit Format Run Options Window Help

successors = []

for neighbor, distance in graph[node].items():
    if neighbor not in path:
        new_g = g_cost + distance
        new_f = max(new_g + heuristic[neighbor],
                    g_cost + heuristic[node])
        successors.append([neighbor, new_g, new_f])

if len(successors) == 0:
    return None, math.inf

while True:
    successors.sort(key=lambda x: x[2])
    best = successors[0]
    if best[2] > f_limit:
        return None, best[2]
    if len(successors) > 1:
```

Ln: 192 Col: 0
11:16
17-07-2026

Name:- Chetan Mandavkar
Roll No. T093
Subject:- Artificial Intelligence

Sheth L.U.J. And Sir M.V. College

```
AI Practical No. 2.py - C:\Users\IT-46\OneDrive\Documents\AI Practical No. 2.py (3.14.6)
File Edit Format Run Options Window Help

    if len(successors) > 1:
        alternative = successors[1][2]
    else:
        alternative = math.inf

    result, best[2] = rbfs(
        best[0],
        goal,
        best[1],
        min(f_limit, alternative),
        path + [best[0]]
    )

    if result is not None:
        return result, best[1]

start_city = "Ratnagiri"
goal_city = "Mumbai"

path, cost = rbfs(
    start_city,
    goal_city,
    0
)
```

Ln: 192 Col: 0

```
AI Practical No. 2.py - C:\Users\IT-46\OneDrive\Documents\AI Practical No. 2.py (3.14.6)
File Edit Format Run Options Window Help

    )
    path + [best[0]]

    if result is not None:
        return result, best[1]

start_city = "Ratnagiri"
goal_city = "Mumbai"

path, cost = rbfs(
    start_city,
    goal_city,
    0,
    math.inf,
    [start_city]
)

print("==== Recursive Best-First Search =====")
print("Route Found :", " -> ".join(path))
print("Total Distance :", cost, "km")
```

Ln: 192 Col: 0

Name:- Chetan Mandavkar

Roll No. T093

Subject:- Artificial Intelligence

Sheth L.U.J. And Sir M.V. College

OUTPUT:-

```
IDLE Shell 3.14.6
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:\Users\IT-46\OneDrive\Documents\AI Practical No. 2.py =====
===== Recursive Best-First Search =====
Route Found : Ratnagiri -> Chiplun -> Mahad -> Panvel -> Mumbai
Total Distance : 130 km
>>>
```

Graph Plotting:-

```
*AI Practical No. 2.py - C:\Users\IT-46\OneDrive\Documents\AI Practical No. 2.py (3.14.6)*
File Edit Format Run Options Window Help

# -----
# GRAPH PLOTTING
# -----

import matplotlib.pyplot as plt
import networkx as nx

node_positions = {
    "Ratnagiri": (2, 8),
    "Chiplun": (2, 6),
    "Mahad": (1, 4),
    "Satara": (3, 4),
    "Panvel": (2, 2),
    "Mumbai": (2, 0)
}

G = nx.DiGraph()

for city, neighbors in graph.items():
    for neighbor, distance in neighbors.items():
        G.add_edge(city, neighbor, weight=distance)

plt.figure(figsize=(10, 8))
```

Name:- Chetan Mandavkar

Roll No. T093

Subject:- Artificial Intelligence

Sheth L.U.J. And Sir M.V. College

```
*AI Practical No. 2.py - C:\Users\IT-46\OneDrive\Documents\AI Practical No. 2.py (3.14.6)*
File Edit Format Run Options Window Help

plt.figure(figsize=(10, 8))

path_edges = list(zip(path, path[1:]))

normal_edges = [edge for edge in G.edges() if edge not in path_edges]

nx.draw_networkx_nodes(
    G,
    node_positions,
    node_size=2600,
    node_color="lightblue"
)

nx.draw_networkx_edges(
    G,
    node_positions,
    edgelist=normal_edges,
    width=1.5,
    edge_color="gray",
    arrows=True
)
```

Ln: 373 Col: 0
11:25
17-07-2026

```
*AI Practical No. 2.py - C:\Users\IT-46\OneDrive\Documents\AI Practical No. 2.py (3.14.6)*
File Edit Format Run Options Window Help

arrows=True,
arrowsize=18
)

nx.draw_networkx_edges(
    G,
    node_positions,
    edgelist=path_edges,
    width=3.5,
    edge_color="darkorange",
    arrows=True,
    arrowsize=18
)

node_labels = {
    city: f"{city}\nh(n)={heuristic[city]}"
    for city in G.nodes()
}

nx.draw_networkx_labels(
    G,
    node_positions,
    labels=node_labels
)
```

Ln: 373 Col: 0
11:25
17-07-2026

Name:- Chetan Mandavkar

Roll No. T093

Subject:- Artificial Intelligence

Sheth L.U.J. And Sir M.V. College

```
*AI Practical No. 2.py - C:\Users\IT-46\OneDrive\Documents\AI Practical No. 2.py (3.14.6)*
File Edit Format Run Options Window Help
labels=node_labels,
font_size=9,
font_weight="bold"
)
|
edge_labels = nx.get_edge_attributes(G, "weight")

formatted_edge_labels = {
    edge: f"{distance} km"
    for edge, distance in edge_labels.items()
}

nx.draw_networkx_edge_labels(
    G,
    node_positions,
    edge_labels=formatted_edge_labels,
    font_color="red",
    font_size=9
)

plt.title(
    "RBFS Routing Map: Ratnagiri to Mumbai\n"
    "(Highlighted Orange Path = Optimal Route)",
    fontsize=14.
)

Ln: 373 Col: 0
11:26
17-07-2026
```

```
*AI Practical No. 2.py - C:\Users\IT-46\OneDrive\Documents\AI Practical No. 2.py (3.14.6)*
File Edit Format Run Options Window Help
edge_labels = nx.get_edge_attributes(G, "weight")

formatted_edge_labels = {
    edge: f"{distance} km"
    for edge, distance in edge_labels.items()
}

nx.draw_networkx_edge_labels(
    G,
    node_positions,
    edge_labels=formatted_edge_labels,
    font_color="red",
    font_size=9
)

plt.title(
    "RBFS Routing Map: Ratnagiri to Mumbai\n"
    "(Highlighted Orange Path = Optimal Route)",
    fontsize=14,
    fontweight="bold"
)

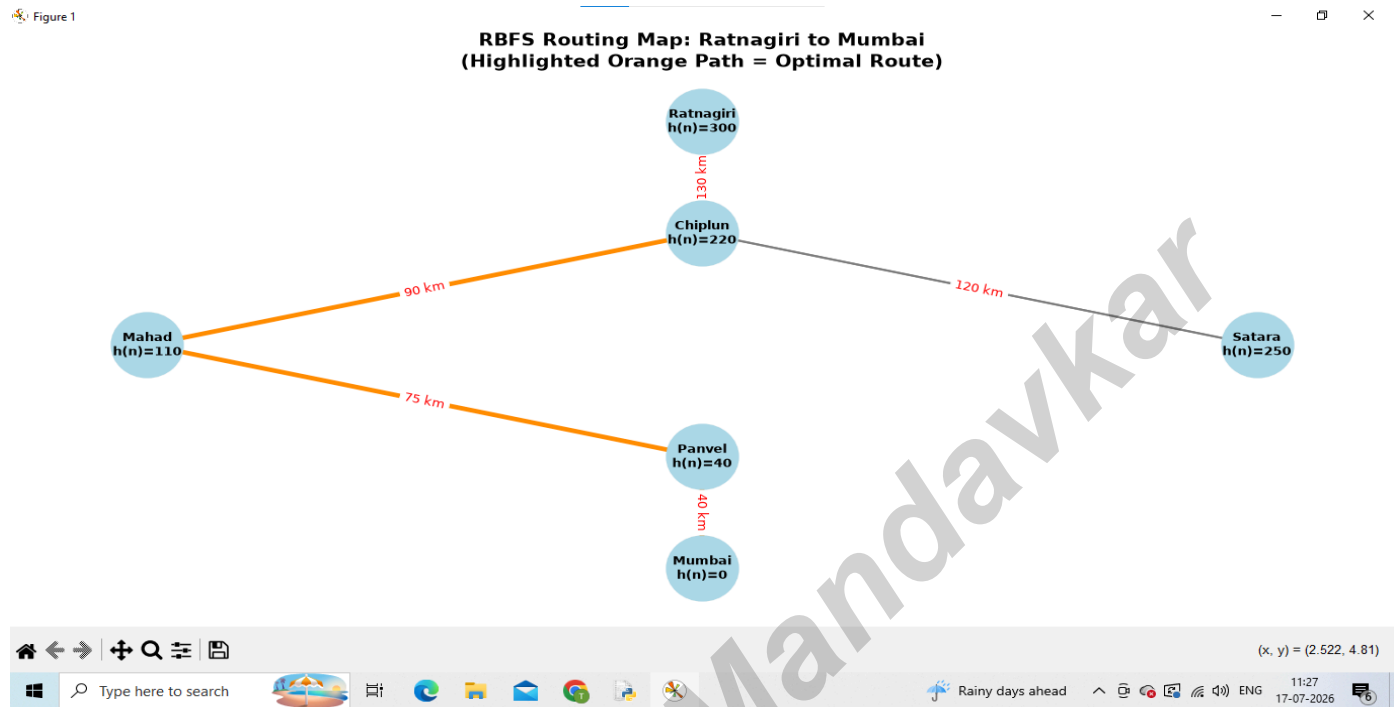
plt.axis("off")
plt.tight_layout()
plt.show()

Ln: 373 Col: 0
11:26
17-07-2026
```

Name:- Chetan Mandavkar
Roll No. T093
Subject:- Artificial Intelligence

Sheth L.U.J. And Sir M.V. College

OUTPUT:-



C] Compare the performance and effectiveness of both algorithms.

Feature	A* Search	Recursive Best-First Search (RBFS)
Search Strategy	Best-First Search using Priority Queue	Recursive Best-First Search

Name:- Chetan Mandavkar

Roll No. T093

Subject:- Artificial Intelligence

Sheth L.U.J. And Sir M.V. College

Evaluation Function	$f(n) = g(n) + h(n)$	$f(n) = g(n) + h(n)$
Heuristic Used	Yes	Yes
Memory Usage	High (stores all generated nodes)	Low (stores only the current search path)
Time Performance	Faster because it avoids repeated exploration	Slightly slower due to backtracking
Space Complexity	Higher	Lower
Optimal Solution	Yes (with an admissible heuristic)	Yes (with an admissible heuristic)
Suitable For	Small and medium-sized road maps	Large maps with limited memory
Path Found in Practical	Mumbai → Lonavala → Pune → Satara	Ratnagiri → Chiplun → Mahad → Panvel → Mumbai

Name:- Chetan Mandavkar

Roll No. T093

Subject:- Artificial Intelligence

Sheth L.U.J. And Sir M.V. College

Performance Analysis

A* Search

- Uses a priority queue to always expand the node with the lowest $f(n) = g(n) + h(n)$ value.
- Stores all generated nodes in memory, resulting in higher memory usage.
- Finds the optimal route quickly when a good heuristic is available.
- Best suited for applications where memory is sufficient.

Recursive Best-First Search (RBFS)

- Uses the same evaluation function as A* but explores recursively.
- Requires much less memory because it stores only the current path and necessary alternatives.
- May revisit nodes during backtracking, making it slightly slower than A*.
- Suitable for large search spaces where memory is limited.

Conclusion

In this practical, *both A Search and RBFS successfully found the optimal path** between the selected cities using heuristic information.

- A* Search performed faster because it keeps all promising nodes in memory, allowing it to reach the destination efficiently.
- RBFS used significantly less memory by exploring paths recursively and backtracking when necessary, making it more memory-efficient.
- Therefore, A* is preferred when sufficient memory is available, while RBFS is a better choice for large-scale pathfinding problems where memory is limited.

Name:- Chetan Mandavkar

Roll No. T093

Subject:- Artificial Intelligence